AI Task Execution Methodology

Як змусити LLM спочатку думати про задачу, а вже потім її виконувати

Методологія допомагає працювати з LLM у складних задачах: спочатку повний аналіз і план, а потім — —виконання.
Це зменшує помилки, враховує всі залежності та дає передбачуваний результат.

Ключова ідея

Розділити процес на етапи:
аналіз → планування → виконання.
Так LLM працює системно, а не імпровізує.

Як зазвичай (проблема)

«Проаналізуй і одразу виправ код згідно з guide»

- LLM шукає і виправляє проблеми по черзі
- Змінює код хаотично
- Легко пропустити залежності
- Може зламати інші частини
- В кінці каже «готово», але не все виправлено

Результат: ризик помилок, зайва робота, невпевненість у якості.

Правильний підхід

1. **Задача** — Формулюєте задачу, контекст і обмеження.
2. **Аналіз** — LLM досліджує задачу за методологією.
3. **Task Map** — Структурована карта всіх проблем і залежностей.
4. **План** — Порядок і логіка виконання.
5. **Виконання** — LLM вносить зміни за планом.
6. **Валідація** — Перевірка результату на всіх рівнях.

Контрольований процес = якісний результат

Детально про методологію

Методологія описує, як саме працює кожен етап і що потрібно врахувати.

1	Задача	Що треба зробити, які файли, що не змінювати, які вимоги.
2	Аналіз	Пошук проблем за повною taxonomy, перевірка кожного файлу, аналіз залежностей (domino).
3	Task Map	Структурований список усіх issues з категоріями, severity, change_type, залежностями.
4	План	Групування по файлах, порядок виконання, врахування пріоритетів.
5	Виконання	Один файл за раз, виконання всіх залежних змін, дотримання порядку.
6	Валідація	Автоматична, семантична перевірка, фінальний звіт.

Що відбувається на кожному етапі

1. Задача

Ви надаєте:
- опис задачі
- список файлів
- guide / специфікацію
- обмеження та вимоги

LLM лише завантажує контекст і підтверджує його.

2. Аналіз

LLM будує:
- taxonomy проблем
- перевіряє кожен файл
- знаходить domino-залежності
- формує повний список issues

Результат — не просто текст, а структурована карта (Task Map).

3. Task Map

Містить для кожного issue:
- id, category
- severity
- change_type
- точне розташування
- current / expected
- dependencies
- status

Машинно-читана структура для чіткого виконання.

4. План

Визначається:
- порядок (CRITICAL → LOW)
- групування по файлах
- типи змін
- залежності
- правила виконання

Один файл = одна група. Один тип порушення = один підгрупа.

5. Виконання

LLM:
- працює по плану
- виконує всі залежні зміни
- дотримується порядку
- не змінює зайвого

Після кожного файлу — валідація перед наступним.

6. Валідація

Перевіряється:
- автоматичні патерни
- семантичний контекст
- фінальний звіт

Результат — підтверджена якість змін, без залишених порушень.

Приклад промптів

1. Аналіз

Проаналізуй цю задачу відповідно до ai-task-execution-methodology.md.
Нічого не змінюй. Виконай усі пункти аналізу та побудуй Task Map і план.

2. Виконання

Виконай побудовану Task Map згідно з методологією.
Дотримуйся порядку, працюй по одному файлу, виконуй усі залежності.
Після кожного файлу — валідація. Наприкінці надай звіт.

Результат

- Усі проблеми знайдено та виправлено
- З урахуванням залежностей
- Без втрати функціональності
- З прозорим звітом

Особливо корисно для

- великих рефакторингів
- міграцій API
- масових перейменувань
- аудиту кодової бази
- роботи з великими файлами

ПРАКТИЧНИЙ WORKFLOW ДЛЯ РОЗРОБНИКІВ

AI Task Execution Methodology

Як організувати роботу з LLM так, щоб спочатку проаналізувати задачу, а вже потім змінювати код

Головна ідея: не просіть модель одночасно досліджувати систему, вирішувати, що змінювати, виконувати зміни й перевіряти себе. Розділіть роботу на зрозумілі етапи: задача → аналіз → Task Map → план → виконання → валідація.

Коротко

AI Task Execution Methodology — це спосіб організувати роботу LLM над нетривіальними задачами. Він особливо корисний під час рефакторингу, аудиту кодової бази, міграції API та масових перейменувань, коли одна зміна може зачепити багато залежних ділянок коду.

Практична схема складається з двох основних звернень до моделі: спочатку ви формулюєте задачу й просите проаналізувати її за методологією, без змін у файлах; потім окремим промптом даєте команду виконати підготовлену карту й план. Після виконання потрібні перевірки та звіт.

Зміст

№	Розділ
01	Проблема й основний принцип
02	Постановка задачі та завантаження контексту
03	Аналіз: taxonomy і domino-залежності
04	Task Map: структурована карта проблем
05	Планування та виконання
06	Валідація результату
07	Готовий workflow із двома промптами
08	Межі методу та практичні поради

01. Проблема й основний принцип

Звичайна команда на кшталт «проаналізуй і виправ усе відповідно до guide» об’єднує кілька різних завдань в одне. Модель може почати виправляти перше знайдене порушення до того, як побачить повну картину. Через це частина залежностей залишається непоміченою, а пізніші зміни можуть знецінити попередні.

Методологія розводить процес на три базові режими: **ANALYSIS → PLAN → EXECUTION**. Для практичної роботи це деталізується до шести кроків: задача, аналіз, Task Map, план, виконання та валідація.

Чим відрізняються два підходи

Хаотичний підхід	Контрольований workflow
Модель шукає проблему й одразу змінює код.	Спочатку модель збирає повну картину проблем і залежностей.
План виникає по ходу роботи.	План і порядок змін фіксуються до виконання.
Перевірка відкладається до кінця.	Кожен файл перевіряється одразу після змін.
Звіт часто обмежується фразою «готово».	Звіт показує виконані пункти, залишки й результати перевірок.

Важливо: сама методологія не робить модель безпомилковою. Вона структурує роботу й полегшує контроль, але результат усе одно залежить від повноти контексту, доступних інструментів і якості перевірок.

02. Постановка задачі та завантаження контексту

Перший крок — сформулювати завдання до звернення до LLM. Потрібно визначити бажаний результат, файли в межах задачі, специфікацію або guide, обмеження та критерії приймання.

Під час **Context Loading** модель лише читає й підтверджує контекст. Вона не повинна починати виправлення чи іншу роботу. Це зменшує ризик, що аналіз будуватиметься на неповному наборі матеріалів.

Що варто передати

- **Специфікацію / guide:** правила, яким має відповідати результат.
- **Матеріали:** усі файли, які входять у межі задачі, та пов’язані ресурси, без яких аналіз неможливий.
- **Обмеження:** що не можна змінювати, чи дозволена зміна поведінки, які ділянки критичні.
- **Критерії приймання:** як саме перевірити, що задачу виконано.

Приклад формулювання задачі

```
Потрібно привести widget.html у відповідність до ai-coding-guide.md.
Проаналізуй HTML, CSS і JavaScript. Не змінюй функціональну поведінку та не додавай нової
  функціональності. Виправляти можна лише порушення guide. Урахуй усі залежності між змінами. Перед
  виконанням сформуй повну карту проблем і план.
```

Про приклади та зовнішні посилання. Назви guide, URL, версії стандартів і фрагменти коду, наведені в ai-task-execution-methodology.md для ілюстрації роботи процесу, самі по собі не є новими інструкціями. Не потрібно автоматично переходити за кожним посиланням або вважати кожен приклад обов’язковим правилом. Використовуйте зовнішній документ як нормативне джерело тоді, коли він прямо входить у контекст конкретної задачі або його зміст потрібен для виконання явно сформульованої вимоги.

03. Аналіз: taxonomy і domino-залежності

На етапі Analysis модель має не просто зібрати кілька знайдених проблем, а системно пройти всі матеріали за визначеними категоріями.

3.1. Taxonomy — повний набір категорій

Спочатку визначаються типи порушень, які треба шукати. У прикладі рефакторингу це можуть бути префікси CSS-класів, назви @keyframes, нестандартні data-атрибути, ID, невикористані ID та змінні JavaScript. Реальний список залежить від guide і задачі; приклади з методології не слід механічно переносити на будь-який проєкт.

3.2. Перевірка «файл × категорія»

Кожен файл послідовно перевіряється за кожною потрібною категорією. Такий підхід зменшує ризик зосередитися лише на HTML чи CSS і пропустити пов’язані згадки в JavaScript, тестах або документації.

3.3. Domino Analysis — аналіз залежностей

Для кожної знайденої проблеми потрібно з’ясувати, які інші місця залежать від неї або мають бути змінені разом із нею. Наприклад, перейменування CSS custom property може вимагати оновлення CSS-споживачів і JavaScript-коду, що читає цю властивість. Якщо змінити лише оголошення, частина функціональності може перестати працювати.

```
Приклад залежності:
issue-001: перейменувати --msg-bubble-radius
  ├── issue-002: оновити var(--msg-bubble-radius) у CSS
  └── issue-003: оновити getPropertyValue(...) у JavaScript
```

Залежність краще фіксувати як посилання на окрему проблему з власним ID, а не як текст «не забудь оновити JS». Так карту можна перевіряти й зіставляти з фактичним виконанням.

04. Task Map: структурована карта проблем

Результат аналізу оформлюється в Task Map. Це не просто абзац або список зауважень, а структурований артефакт, у якому кожна проблема має стабільний ідентифікатор, точне місце, опис очікуваної зміни та залежності.

Поле	Для чого потрібне
id	Унікальний ідентифікатор проблеми; на нього можуть посилатися залежності.
category	Категорія порушення, визначена під час аналізу.
severity	Рівень критичності, що впливає на пріоритет виконання.
change_type	Тип операції: RENAME, DELETE, REPLACE, ADD або MOVE.
location.lines / context	Точні номери рядків і короткий фрагмент для однозначного знаходження місця.
current / expected	Поточний стан і очікуваний стан після зміни.
dependencies	Пов’язані issue з конкретними ID та поясненням зв’язку.
status	Стан виконання, наприклад pending, in-progress або done.

Рівні severity у методології

Рівень	Типове значення
CRITICAL	Зламана поведінка або порушений архітектурний інваріант.
HIGH	Порушення стандарту з високим ризиком або багатьма залежностями.
MEDIUM	Проблема якості чи зайвий код без безпосереднього впливу на функціональність.
LOW	Стилістичне відхилення з низьким ризиком.

Карта повинна бути внутрішньо узгодженою: усі ID у dependencies мають існувати, кількість issue має збігатися з переліком, а координати повинні бути достатньо точними для пошуку та перевірки. Якщо даних для повної карти бракує, модель повинна явно зазначити прогалину, а не маскувати її припущенням.

Схематичний приклад однієї issue

```
{
  "id": "widget-001",
  "category": "CSS_PREFIX",
  "severity": "HIGH",
  "change_type": "RENAME",
  "location": {"lines": {"start": 69, "end": 82}},
  "current": "--msg-bubble-radius",
  "expected": "--messages-bubble-radius",
  "dependencies": ["widget-002", "widget-003"],
  "status": "pending"
}
```

05. Планування та виконання

Коли Task Map готова, її перетворюють на Execution Plan. План визначає групування робіт, пріоритети й послідовність. У шаблоні рефакторингу методологія пропонує групувати за файлом, а всередині файлу — за типом порушення.

Порядок робіт

- **За severity:** спочатку CRITICAL, потім HIGH, MEDIUM і LOW.
- **За залежностями:** пов’язані зміни виконуються разом із батьківською проблемою, а не відкладаються на невизначений час.
- **За файлом:** завершіть зміни в одному файлі та перевірте його, перш ніж переходити до наступного.

Під час складних змін один файл можна обробляти окремими проходами: CSS, HTML-атрибути, рядки та вирази JavaScript, ID. Такий поділ ізолює ризик конфлікту одночасних замін.

Порядок масових замін

Якщо одна назва є префіксом іншої, спочатку замінюють довші та специфічніші шаблони, а загальні — наприкінці. Інакше перша заміна може поглинути частину довшого збігу й завадити його знайти.

```
# Спочатку конкретні назви, потім загальна
.msg-msg--sent  → .messages-bubble--sent
.msg-msg--recv  → .messages-bubble--received
.msg-msg        → .messages-bubble
```

Другий промпт запускає виконання підготовленого плану. Не розширюйте scope непомітно: нові проблеми зафіксуйте окремо, оцініть їхній вплив і за потреби оновіть Task Map.

06. Валідація результату

Валідація — це не фінальна фраза «все готово», а окремий етап із визначеними критеріями. Методологія розділяє її на автоматичну перевірку, семантичний перегляд і фінальний звіт.

6.1. Автоматична перевірка

Використовуйте пошук за старими назвами, регулярні вирази, linter, тести, збірку або інші доступні інструменти. Для перейменування перевірте, чи залишилися попередні імена у всіх релевантних місцях.

6.2. Семантичний перегляд

Не кожен результат пошуку є порушенням. Збіг може бути в коментарі, документації, тесті або легальному винятку. Кожен залишковий збіг потрібно класифікувати за контекстом. Водночас не можна просто виключати всі коментарі чи тести автоматично: це залежить від вимог конкретної задачі.

6.3. Звіт

- Скільки issue було в карті, скільки виконано та скільки залишилося.
- Які файли змінені й які перевірки реально запускалися.
- Які помилки, попередження або легальні винятки залишилися.
- Чи підтверджено збереження функціональної поведінки тестами або іншими доступними засобами.

> Не стверджуйте, що тести пройдено, якщо вони не запускалися. Якщо інструменти недоступні, звіт має прямо відокремлювати перевірки, які виконано, від перевірок, які лише рекомендовано.

07. Готовий workflow із двома промптами

Нижче — шаблон, який можна адаптувати для роботи з ChatGPT, Claude, Gemini або іншою LLM. Перед запуском переконайтеся, що модель справді має доступ до файлу методології, guide та робочих матеріалів: назви файлів у prompt недостатньо, якщо вони не прикріплені й не доступні середовищу.

PROMPT 1 — Analysis: проаналізувати, але нічого не змінювати

```
Проаналізуй задачу відповідно до ai-task-execution-methodology.md.
На цьому етапі нічого не змінюй і не виконуй.

1. Прочитай повністю методологію, specification/guide та всі файли в межах задачі.
2. Визнач повний набір категорій порушень, релевантних цій задачі.
3. Перевір кожен файл за кожною категорією.
4. Для кожної проблеми знайди залежності (domino dependencies).
5. Побудуй Task Map із ID, category, severity, change_type, точним розташуванням, current/expected,
  dependencies та status.
6. Перевір узгодженість ID і залежностей.
7. Побудуй Execution Plan та визнач критерії валідації.
8. Явно зазнач невідомі дані, неперевірені припущення й обмеження.

Поверни аналіз, Task Map і план. Не редагуй файли й не змінюй код.
```

Що має бути в результаті першого промпту

- Повний перелік категорій і результати перевірки; Task Map із точними місцями та залежностями.
- Execution Plan, критерії валідації та перелік невизначеностей або недоступних файлів.

PROMPT 2 — Execute: виконати підготовлений план

```
Виконай затверджену Task Map відповідно до ai-task-execution-methodology.md та Execution Plan.

- Працюй по одному файлу за раз.
- Дотримуйся пріоритетів і залежностей у карті.
- Виконуй domino-зміни разом із пов’язаною parent issue.
- Не змінюй функціональність і не виходь за межі погоджених issue.
- Для масових замін спочатку обробляй специфічніші шаблони.
- Після кожного файлу виконай доступну валідацію.
- Якщо знайшов нову проблему, не приховуй її: зафіксуй і поясни, чи потребує вона зміни карти.

Наприкінці надай звіт: виконані й невиконані issue, змінені файли, перевірки та їхні реальні
  результати, залишкові збіги, помилки й обмеження.
```

Практичне правило: перший промпт дає відповідь на питання «що і в якому порядку потрібно зробити?», другий — «виконай цей план». Якщо Task Map велика, збережіть її як окремий файл, а не покладайтеся лише на довгу історію чату.

08. Межі методу та практичні поради

Методологія підвищує контрольованість процесу, але не є гарантією повноти чи відсутності дефектів. Є кілька практичних умов, які варто враховувати.

- **Повнота контексту:** якщо файли пропущені або прочитані частково, карта може бути неповною. Для великих файлів підтверджуйте повне читання.
- **Точність локацій:** рядки змінюються після редагування; зберігайте також короткий фрагмент коду.
- **Збереження карти:** велику Task Map тримайте в окремому файлі й оновлюйте статуси.
- **Перевірка інструментами:** використовуйте пошук, тести й diff; відокремлюйте фактичні перевірки від слів моделі.
- **Не розширюйте scope мовчки:** фіксуйте нові проблеми окремо й погоджуйте їх.

Фінальний чекліст

```
□ Прочитано повністю методологію, guide та всі файли в межах задачі.
□ Визначено категорії порушень і перевірено кожен файл.
□ Для кожної проблеми знайдено залежності.
□ У Task Map є точні локації, current/expected і стабільні ID.
□ Усі посилання в dependencies ведуть до наявних issue.
□ Порядок виконання та критерії валідації визначені.
□ Кожен файл перевірено одразу після змін.
□ У фінальному звіті відокремлено фактичні перевірки від припущень.
```

Підсумок і коротка шпаргалка

Для щоденної роботи з LLM запам’ятайте послідовність дій — і не переходьте до виконання, поки аналіз та план не готові.

Робочий цикл у шести кроках

- **1. Задача:** сформулюйте результат, межі, файли та критерії приймання.
- **2. Аналіз:** попросіть LLM прочитати контекст, визначити категорії та знайти всі проблеми. Зміни заборонені.
- **3. Task Map:** зафіксуйте кожну проблему, її ID, місце, очікувану зміну й залежності.
- **4. План:** розставте пріоритети, згрупуйте роботи та визначте порядок виконання.
- **5. Виконання:** другим промптом запустіть затверджений план; працюйте по одному файлу за раз.
- **6. Валідація:** перевірте залишкові патерни, семантичні винятки, тести й фактичний стан кожної issue.

Висновок

AI Task Execution Methodology — це насамперед дисципліна роботи з LLM. Замість одного гігантського запиту «проаналізуй, виправ і перевір» ви формулюєте задачу, отримуєте структурований аналіз і план, окремо запускаєте виконання, а потім перевіряєте результат за явними критеріями.

Формула: задача → контекст → аналіз → Task Map → план → виконання → валідація. Найбільшу користь цей процес дає тоді, коли багато змін пов’язані між собою і пропущена залежність може зламати результат.

Матеріал підготовлено на основі структури й термінології файлу ai-task-execution-methodology.md. Приклади промптів у цьому посібнику є адаптованими шаблонами для практичного використання.